

A Comparative Study of Three Algorithms for Mining Top-K High Utility Itemsets from Massive Datasets with Parallelism Analysis

Le Anh Tuan, To Hoang Dao

Ton Duc Thang University (TDTU), Ho Chi Minh City, Vietnam

Abstract. High Utility Itemset Mining (HUIM) is a fundamental task in data mining that discovers itemsets yielding high profit from transactional databases. Unlike traditional frequent itemset mining, HUIM considers both the quantity and unit profit of items, making it significantly more challenging due to the loss of the downward closure property. This paper presents a comparative study of three algorithms for mining top- k high utility itemsets: **TKU** (two-phase, UP-Tree-based), **TKO** (one-phase, utility-list-based), and **THUI** (one-phase, utility-list-based with LIU matrix). We analyze the theoretical foundations, map each algorithmic step to its corresponding Go implementation, and present experimental results comparing runtime, memory consumption, and scalability. Furthermore, we identify parallelization opportunities in each algorithm using techniques such as MapReduce and Fork/Join, and propose concrete strategies for improving performance on massive datasets.

Keywords: High Utility Itemset Mining · Top-K Mining · MapReduce · Fork/Join · Parallelism · SPMF · Massive Datasets

1 Introduction

Traditional frequent itemset mining (FIM) assumes all items have equal importance and uses a binary model (present or absent). However, in real-world retail analytics, items have different unit profits and appear in varying quantities. **High Utility Itemset Mining (HUIM)** [3] addresses this limitation by assigning external utilities (unit profit) and internal utilities (purchase quantity) to items.

A major challenge in HUIM is setting the minimum utility threshold (`min_util`). Setting it too low produces an overwhelming number of results; setting it too high yields none. The **top- k** formulation [1] elegantly solves this by asking the user to specify only the desired number of results k , while the algorithm automatically determines the appropriate threshold.

In this paper, we study and compare three representative algorithms:

1. **TKU** [1]: A two-phase algorithm using UP-Tree with five threshold-raising strategies (PE, NU, MD, MC, SE).

2. **TKO** [1]: A one-phase algorithm using utility-lists with strategies RUC, RUZ, and EPB.
3. **THUI** [2]: A one-phase algorithm using utility-lists enhanced with a Leaf Itemset Utility (LIU) matrix for more effective threshold raising.

All three algorithms aim to mine top- k high utility itemsets but employ fundamentally different data structures and pruning strategies. The remainder of this paper is organized as follows: Section 2 defines the problem formally. Section 3 details each algorithm with code-to-theory mapping. Section 4 presents experimental comparisons. Section 5 analyzes parallelization opportunities. Section 6 concludes.

2 Preliminaries and Problem Definition

2.1 Basic Definitions

Let $I = \{I_1, I_2, \dots, I_m\}$ be a set of distinct items and $D = \{T_1, T_2, \dots, T_n\}$ be a transactional database. Each transaction $T_r \in D$ is a subset of I . Each item I_j has an **external utility** (unit profit) $p(I_j)$ and an **internal utility** (quantity) $q(I_j, T_r)$ in transaction T_r .

Definition 1 (Utility of an Item). *The utility of item I_j in transaction T_r is:*

$$EU(I_j, T_r) = p(I_j) \times q(I_j, T_r) \quad (1)$$

Definition 2 (Utility of an Itemset). *The utility of an itemset X in the database is:*

$$EU(X) = \sum_{T_r \in D \wedge X \subseteq T_r} \sum_{I_j \in X} EU(I_j, T_r) \quad (2)$$

Definition 3 (Transaction Utility). *The transaction utility of T_r is:*

$$TU(T_r) = \sum_{I_j \in T_r} EU(I_j, T_r) \quad (3)$$

Definition 4 (Transaction-Weighted Utilization). *The TWU of an itemset X is:*

$$TWU(X) = \sum_{X \subseteq T_r \wedge T_r \in D} TU(T_r) \quad (4)$$

2.2 Key Properties

Theorem 1 (Transaction-Weighted Downward Closure – TWDC). *For any itemsets $X \subseteq Y$: $TWU(Y) \leq TWU(X)$. Furthermore, $EU(X) \leq TWU(X)$.*

This property is critical because, unlike $EU(X)$, $TWU(X)$ is anti-monotone. If $TWU(X) < \text{min_util}$, then all supersets of X can be safely pruned.

Definition 5 (Minimum Item Utility – MIU).

$$MIU(X) = \left(\sum_{I_j \in X} \min_{T_r \supseteq X} \{EU(I_j, T_r)\} \right) \times SC(X) \quad (5)$$

where $SC(X)$ is the support count of X . $MIU(X)$ is a **lower bound** of $EU(X)$, used to safely raise the threshold.

Definition 6 (Utility-List). For an itemset X , its utility-list $UL(X)$ contains one entry $(tid, iutil, rutil)$ per transaction T_r where $X \subseteq T_r$:

- tid : transaction identifier
- $iutil = EU(X, T_r)$: the exact utility of X in T_r
- $rutil$: sum of utilities of items ordered after X in T_r

Key property: $EU(X) = \sum_{e \in UL(X)} e.iutil$ gives the exact utility directly.

2.3 Problem Statement

Top- k High Utility Itemset Mining: Given a transactional database D with utility information and an integer k , discover the k itemsets with the highest utility values without requiring a pre-defined minimum utility threshold.

3 Algorithm Analysis with Code-to-Theory Mapping

All three algorithms have been re-implemented in Go. We present each algorithm with pseudocode derived from the actual Go source code, with source file references (package path and line numbers) in each caption.

3.1 Algorithm 1: TKU (Two-Phase, Tree-Based)

TKU [1] operates in two phases. Phase 1 builds a compressed UP-Tree and generates candidate itemsets (PKHUIs) using five threshold-raising strategies. Phase 2 verifies candidates by scanning the original database to compute exact utilities.

Phase 1: Candidate Generation via UP-Tree

Step 1 – Database Scan and TWU Computation. The first database scan computes TWU for each item, and the minimum/maximum item utilities (miu , mau) needed by later pruning strategies.

Algorithm 1 TWU, MIU, MAU Computation — First DB Scan (Ref: tku/tku.go, preEvaluation, lines 216–269)

Require: Database D

Ensure: $TWU[\cdot]$, $MinUtil[\cdot]$, $MaxUtil[\cdot]$ for each item

```

1: for each transaction  $T_r \in D$  do
2:   Parse  $T_r$  into items  $\{I_1, \dots, I_w\}$  and utilities  $\{u_1, \dots, u_w\}$ 
3:    $TU \leftarrow$  transaction utility of  $T_r$ 
4:   for each item  $I_j$  in  $T_r$  with utility  $u_j$  do
5:     if  $MinUtil[I_j] = 0$  or  $MinUtil[I_j] > u_j$  then
6:        $MinUtil[I_j] \leftarrow u_j$   $\triangleright$  Track minimum item utility
7:     end if
8:     if  $MaxUtil[I_j] < u_j$  then
9:        $MaxUtil[I_j] \leftarrow u_j$   $\triangleright$  Track maximum item utility
10:    end if
11:     $TWU[I_j] \leftarrow TWU[I_j] + TU$   $\triangleright$  Accumulate TWU
12:  end for
13: end for

```

Theory $\rightarrow$ Code: This implements Definition 4 (TWU) and collects $miu(I_j) = \min_{T_r} \{EU(I_j, T_r)\}$ used by strategies MD and MC.

Step 2 – Strategy PE (Pre-Evaluation). During the first scan, TKU also builds a Pre-Evaluation Matrix (PEM) using only the first item of each transaction paired with subsequent items:

$$PEM[I_1][I_i] += EU(\{I_1, I_i\}, T_r) \quad (6)$$

The k -th largest value in PEM safely raises `min_util_Border` from 0, since PEM values are lower bounds.

Algorithm 2 PE Strategy — Pre-Evaluation Matrix (Ref: tku/tku.go, preEvaluation+getInitialUtility, lines 216–311)

```

1: for each transaction  $T_r = \{I_1, I_2, \dots, I_w\}$  do
2:   for  $i \leftarrow 2$  to  $w$  do
3:      $PEM[I_1][I_i] \leftarrow PEM[I_1][I_i] + EU(I_1, T_r) + EU(I_i, T_r)$ 
4:   end for
5: end for
6:  $minUtil \leftarrow k$ -th largest value in  $PEM$ 

```

Step 3 – UP-Tree Construction. Transactions are filtered (items with $TWU < \text{min_util_Border}$ removed), sorted by TWU descending, and inserted into the UP-Tree. Each node stores: item name, count, and node utility (Nu). The UP-Tree compresses n transactions into shared-prefix paths, reducing memory from $O(n \cdot w)$ to $O(|\text{tree}|)$.

Step 4 – Strategy MD (MIU of Descendants). After the tree is built, TKU examines parent-descendant pairs to compute lower bounds:

$$MIU(\{\alpha, \beta\}) = (miu(\alpha) + miu(\beta)) \times SC_{\text{tree}}(\alpha, \beta) \quad (7)$$

Algorithm 3 MD Strategy — MIU of Descendants (Ref: tku/tku.go, runPhase1CandidateGeneration, lines 126–137)

```

1: for each child node  $c$  of  $tree.root$  do
2:    $SumDS[\cdot] \leftarrow$  count of each item among descendants of  $c$ 
3:    $\alpha \leftarrow c.item$ 
4:   for each item  $\beta$  where  $SumDS[\beta] \neq 0$  and  $\beta \neq \alpha$  do
5:      $value \leftarrow (miu[\alpha] + miu[\beta]) \times SumDS[\beta]$ 
6:     UPDATETOPKHEAP( $DSHeap, value$ ) ▷ Raise threshold
7:   end for
8: end for

```

Step 5 – UPGrowth with Strategy MC (MIU of Candidates). The recursive UPGrowth procedure traverses the tree FP-Growth-style. For each candidate X generated, MC computes $MIU(X)$ and inserts it into a size- k min-heap. When the heap is full, its minimum becomes the new threshold.

Algorithm 4 MC Strategy — MIU of Candidates (Ref: tku/tku.go, updateNodeCountHeap, lines 378–393)

Require: Candidate X with $SumMiu$, support count SC

```

1:  $MIU \leftarrow SumMiu \times SC$ 
2: if  $MIU > globalMinUtil$  then
3:   UPDATETOPKHEAP( $CandidateHeap, MIU$ )
4: end if

```

Safety guarantee: Since $MIU(X) \leq EU(X)$, raising the threshold using MIU values never misses true top- k results.

Phase 2: Exact Utility Verification with Strategy SE Phase 2 loads the entire original database into RAM as two parallel arrays ($HDB[]$ for items, $BNF[]$ for utilities), then verifies each candidate:

Algorithm 5 Phase 2 — Exact Utility Verification (Ref: `tku/phase2.go`, `readCandidateItemsets`, lines 125–193)

Require: Sorted candidates $\mathcal{C}$ (descending by estimated utility), database D

Ensure: Top- k HUIs with exact utilities

```

1: for each candidate  $X \in \mathcal{C}$  do
2:   if  $X.estUtil < minUtility$  then
3:     skip  $X$  ▷ SE: early termination
4:   end if
5:    $exactUtil \leftarrow 0$ 
6:   for each transaction  $T_r \in D$  do
7:     if  $X \subseteq T_r$  then
8:        $exactUtil \leftarrow exactUtil + EU(X, T_r)$  ▷ Sum exact utilities
9:     end if
10:  end for
11:  if  $exactUtil \geq minUtility$  then
12:    Insert  $(X, exactUtil)$  into Top- $k$  Heap
13:    Update  $minUtility$  from heap minimum
14:  end if
15: end for

```

Strategy SE: Candidates are sorted in descending order by estimated utility before verification. Strong candidates verified first rapidly raise $minUtility$, allowing weaker candidates to be skipped.

Complexity: Phase 2 is $O(C' \times N \times L)$ where C' is candidates verified, N transaction count, L average candidate length. This is TKU's main bottleneck.

3.2 Algorithm 2: TKO (One-Phase, Utility-List-Based)

TKO [1] eliminates Phase 2 by using **utility-lists** for direct exact utility computation during mining.

Utility-List Construction and Join To mine itemset $X_{ab} = X_a \cup X_b$, TKO joins the utility-lists of X_a and X_b :

Algorithm 6 Utility-List Join for TKO**Require:** Utility-lists $UL(X_a)$, $UL(X_b)$, prefix $UL(P)$ **Ensure:** Utility-list $UL(X_{ab})$

```

1: for each entry  $e_a \in UL(X_a)$  do
2:   if  $\exists e_b \in UL(X_b)$  s.t.  $e_a.tid = e_b.tid$  then
3:     if  $P \neq \emptyset$  then
4:       Find  $e_p \in UL(P)$  s.t.  $e_p.tid = e_a.tid$ 
5:        $iutil \leftarrow e_a.iutil + e_b.iutil - e_p.iutil$ 
6:     else
7:        $iutil \leftarrow e_a.iutil + e_b.iutil$ 
8:     end if
9:     Add  $(e_a.tid, iutil, e_b.rutil)$  to  $UL(X_{ab})$ 
10:  end if
11: end for

```

Key advantage: $EU(X_{ab}) = \sum_{e \in UL(X_{ab})} e.iutil$ — exact utility is available immediately, no second database scan needed.

Pruning Strategies

RUC (Raising by Utility of Candidates). Whenever $EU(X) \geq \text{min_util_Border}$, insert X into a size- k min-heap. When full, raise the threshold to the heap's minimum.

RUZ (Reducing by Z-elements). If extension item z has $rutil = 0$ in all utility-list entries, the upper bound $\sum(iutil + rutil)$ is tightened, enabling stronger pruning.

EPB (Exploring Promising Branches first). Extensions are sorted by $\sum(iutil + rutil)$ descending. By exploring strongest branches first, TKO discovers high-utility itemsets early and raises the threshold faster.

Pruning condition: If $\sum_{e \in UL(X)} (e.iutil + e.rutil) < \text{min_util_Border}$, prune all supersets of X .

3.3 Algorithm 3: THUI (One-Phase, Utility-List + LIU Matrix)

THUI [2] is a one-phase algorithm that, like TKO, uses utility-lists, but introduces a novel **Leaf Itemset Utility (LIU)** matrix to achieve more aggressive threshold raising, particularly by leveraging information about leaf-level (longest) itemsets.

Overview and Key Innovation THUI's main contribution is that it computes utility information for **leaf itemsets** — itemsets corresponding to complete transaction suffixes in the search tree — during the database scan phase. This yields tighter lower bounds for the threshold *before* the recursive mining even begins.

Step 1 – RIU: Raising by Real Item Utilities During the first scan, THUI computes the TWU and the **real item utility** (RIU) for each single item:

$$RIU(I_j) = \sum_{T_r \supseteq \{I_j\}} EU(I_j, T_r) \quad (8)$$

The k -th largest RIU value is used to raise the initial threshold.

Algorithm 7 RIU Computation — First DB Scan (Ref: `thui/thui.go`, `firstScan`, lines 315–352)

Require: Database D , parameter k

Ensure: Initial $minUtility$ raised by RIU

```

1: for each transaction  $T_r \in D$  do
2:    $TU \leftarrow$  transaction utility of  $T_r$ 
3:   for each item  $I_j$  in  $T_r$  with utility  $u_j$  do
4:      $TWU[I_j] \leftarrow TWU[I_j] + TU$  ▷ TWU accumulation
5:      $RIU[I_j] \leftarrow RIU[I_j] + u_j$  ▷ Real item utility
6:   end for
7: end for
8: Sort items by  $RIU$  in descending order
9: if number of items  $\geq k$  then
10:    $minUtility \leftarrow k$ -th largest  $RIU$  value
11: end if

```

Theory → Code: Since $RIU(I_j) = EU(\{I_j\})$ is the exact utility of a single-item set, and any top- k result has $EU \geq$ the k -th largest single-item utility, this is a safe lower bound.

Step 2 – Utility-List Construction with LIU Matrix During the second scan, THUI builds utility-lists identically to TKO/HUI-Miner. Additionally, it constructs two auxiliary structures:

EUCS Map (optional). For each pair of items (I_a, I_b) , stores accumulated TWU and real 2-item utility. Used similarly to TKO’s EUCS pruning.

Algorithm 8 EUCS Pairwise Utility Update (Ref: `thui/thui.go`, `updateEUCSprune`, lines 439–458)

Require: Current item pair (I_a, I_b) in revised transaction, $newTWU$

```

1: for each item  $I_b$  after  $I_a$  in the revised transaction do
2:    $EUCS[I_a][I_b].twu \leftarrow EUCS[I_a][I_b].twu + newTWU$ 
3:    $EUCS[I_a][I_b].utility \leftarrow EUCS[I_a][I_b].utility + u_a + u_b$ 
4: end for

```

LIU Matrix (Leaf Itemset Utility). The core innovation of THUI. For each item at the end of a transaction (“leaf”), THUI tracks the cumulative utility of complete suffix sequences:

Algorithm 9 LIU Matrix Update — Leaf Itemset Utility (Ref: `thui/thui.go`, `updateLeafprune`, lines 460–480)

Require: Item I_{leaf} at position i , revised transaction T' , utility-lists ULs

```

1:  $cutil \leftarrow u_{leaf}$  ▷ Cumulative utility starts from leaf
2:  $leafIdx \leftarrow$  index of  $I_{leaf}$  in  $ULs$  (sorted by TWU)
3: for  $j \leftarrow i - 1$  downto 0 do ▷ Walk backwards
4:    $I_{prev} \leftarrow T_j.item$ 
5:    $prevIdx \leftarrow leafIdx - 1$ 
6:   if  $UL_{s_{prevIdx}}.item \neq I_{prev}$  then
7:     break ▷ Not consecutive in sort order
8:   end if
9:    $cutil \leftarrow cutil + u_{prev}$ 
10:   $LIU[leafIdx][prevIdx] \leftarrow LIU[leafIdx][prevIdx] + cutil$ 
11:   $leafIdx \leftarrow prevIdx$ 
12: end for
```

Theory: The LIU matrix captures the utility of *complete consecutive suffix itemsets* in transactions. For a transaction $T = \{a, b, c, d, e\}$ (sorted by TWU), the leaf item is e , and consecutive suffixes like $\{d, e\}$, $\{c, d, e\}$ are tracked. Their accumulated utilities across all transactions provide tight lower bounds.

Step 3 – Threshold Raising via LIU-E and LIU-LB Before recursive mining begins, THUI uses the LIU matrix to raise the threshold with two sub-strategies:

LIU-Exact (LIU-E). Exact utility values stored in the LIU matrix are directly inserted into a size- k priority queue:

Algorithm 10 LIU-Exact Threshold Raising (Ref: `thui/thui.go`, `raisingThresholdLeaf`, lines 686–692)

```

1: for each entry  $(leafIdx, startIdx) \rightarrow value$  in  $LIU$  do
2:   if  $value \geq minUtility$  then
3:      $INSERTTOPKHEAP(leafHeap, value)$  ▷ Raise threshold
4:   end if
5: end for
6: if  $|leafHeap| \geq k$  and  $leafHeap.min > minUtility$  then
7:    $minUtility \leftarrow leafHeap.min$ 
8: end if
```

LIU-Lower Bound (LIU-LB). THUI generates combinatorial lower bounds by subtracting subsets of items from known leaf utilities:

Algorithm 11 LIU-LB Threshold Raising (Ref: `thui/thui.go`, `raisingThresholdLeaf`, lines 694–721)

Require: Leaf entry with range $[st, end)$ and total utility V

```

1: for  $i \leftarrow st + 1$  to  $end - 2$  do                                ▷ Remove one item
2:    $V_1 \leftarrow V - EU(ULs[i])$ 
3:   if  $V_1 \geq minUtility$  then
4:     INSERTTOPKHEAP( $leafHeap, V_1$ )
5:   end if
6:   for  $j \leftarrow i + 1$  to  $end - 2$  do                                ▷ Remove two items
7:      $V_2 \leftarrow V - EU(ULs[i]) - EU(ULs[j])$ 
8:     if  $V_2 \geq minUtility$  then
9:       INSERTTOPKHEAP( $leafHeap, V_2$ )
10:    end if
11:  end for
12: end for

```

Theory: If a leaf itemset $\{c, d, e\}$ has accumulated utility V , then the subset $\{c, e\}$ (removing d) has utility $\geq V - EU(\{d\})$. This provides additional lower bounds without any database scanning.

Step 4 – Recursive Mining (THUI procedure) The core mining is structurally identical to HUI-Miner/TKO — recursive utility-list joins with on-the-fly threshold raising:

Algorithm 12 THUI Recursive Mining (Ref: `thui/thui.go`, `thui`, lines 488–522)

Require: Prefix P , prefix utility-list pUL , extension utility-lists ULs

```

1: for  $i \leftarrow |ULs| - 1$  downto 0 do                                ▷ Check single extensions
2:   if  $ULs_i.sumIutils \geq minUtility$  then
3:      $SAVETOTOPK(P \cup \{ULs_i.item\}, ULs_i)$ 
4:   end if
5: end for
6: for  $i \leftarrow |ULs| - 2$  downto 0 do                                ▷ Generate multi-item extensions
7:    $X \leftarrow ULs_i$ 
8:   if  $X.sumIutils + X.sumRutils \geq minUtility$  then                    ▷ Pruning
9:      $exULs \leftarrow \emptyset$ 
10:    for  $j \leftarrow i + 1$  to  $|ULs| - 1$  do
11:       $Y \leftarrow ULs[j]$ 
12:       $Z \leftarrow CONSTRUCT(pUL, X, Y)$                                 ▷ Join utility-lists
13:      if  $Z \neq null$  then
14:         $exULs \leftarrow exULs \cup \{Z\}$ 
15:      end if
16:    end for
17:     $THUI-MINE(P \cup \{X.item\}, X, exULs)$                             ▷ Recurse
18:  end if
19: end for

```

The `SAVETOTOPK` method maintains a size- k min-heap and raises *minUtility* whenever the heap is full (Ref: `thui/thui.go`, `save`, lines 594–614).

4 Experimental Comparison

4.1 Experimental Setup

All three algorithms (TKU, TKO, THUI) are implemented in Go. We used the standard SPMF input format: `items:TU:utilities`.

4.2 Datasets

Table 1. Dataset characteristics

Dataset	Transaction	Item	Avg Len	Type	Source
Chicago_Crimes_2001_to_2017_utility	2,662,309	35	1.795	Sparse	SPMF
mushroom_utility_SPMF	8,416	119	23	Sparse	SPMF
retail_utility_spmf	88,162	16,470	10.3	Sparse	SPMF
connect_utility_spmf	67,557	129	43	Dense	SPMF
accidents_utility_spmf	340,183	468	33.8	Dense	SPMF

Datasets are available from the SPMF repository [4].

4.3 Results

Dataset: Chicago_Crimes_2001_to_2017_utility https://www.philippe-fournier-viger.com/spmf/publicdatasets/Chicago_Crimes_2001_to_2017_utility.txt

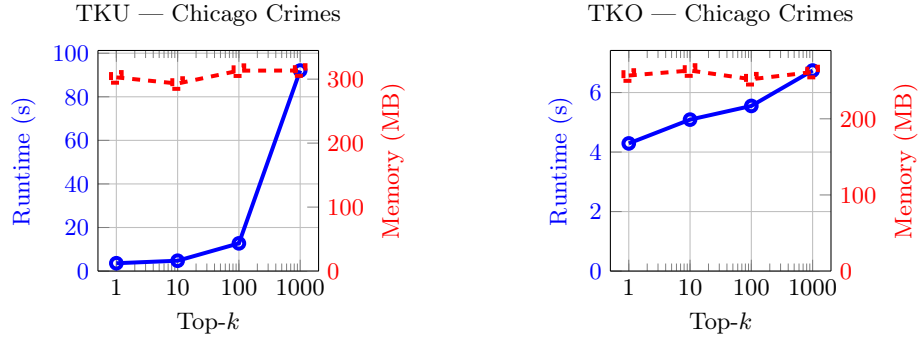


Fig. 1. TKU (left) and TKO (right) runtime and memory on Chicago Crimes dataset. Blue solid line: runtime (left axis). Red dashed line: memory (right axis).

Dataset: mushroom_utility_SPMF https://www.philippe-fournier-viger.com/spmf/publicdatasets/mushroom_utility_SPMF.txt

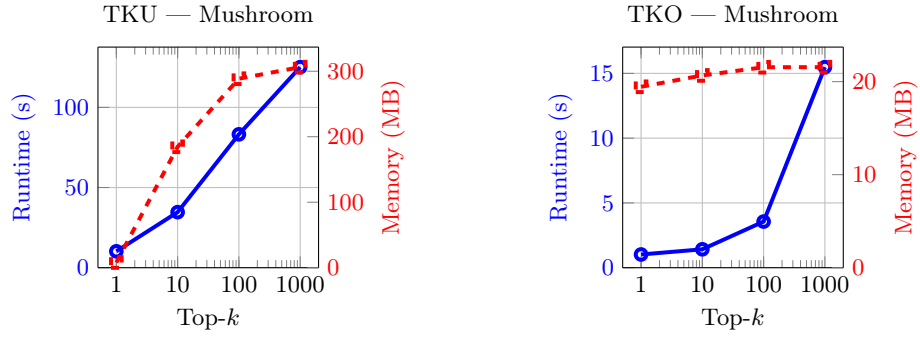


Fig. 2. TKU (left) and TKO (right) runtime and memory on Mushroom dataset.

Dataset: retail_utility_spmf https://www.philippe-fournier-viger.com/spmf/publicdatasets/retail_utility_spmf.txt

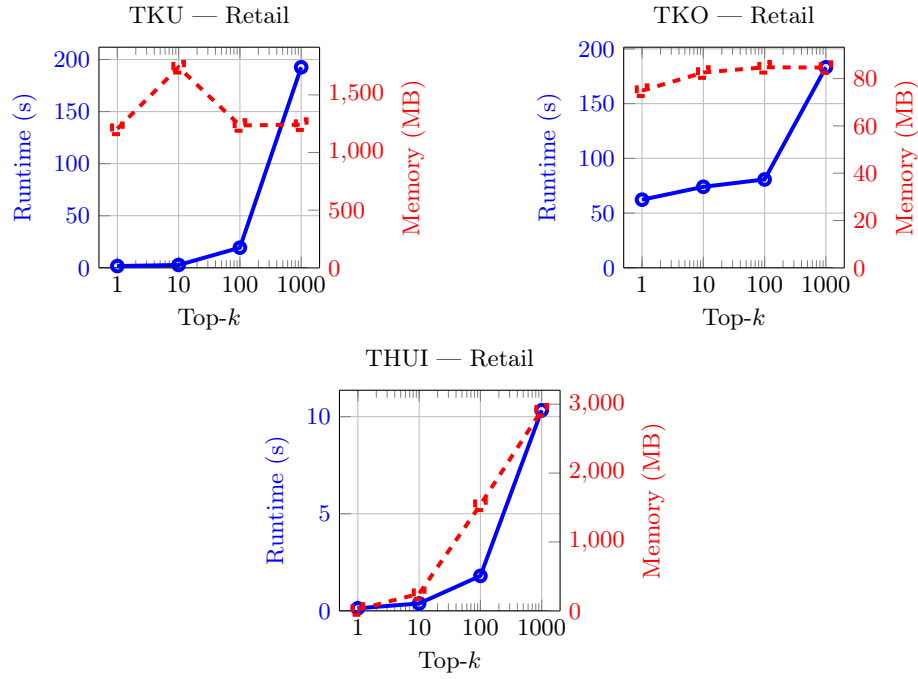


Fig. 3. TKU, TKO, and THUI runtime and memory on Retail dataset.

Dataset: `connect_utility_spmf` https://www.philippe-fournier-viger.com/spmf/publicdatasets/connect_utility_spmf.txt

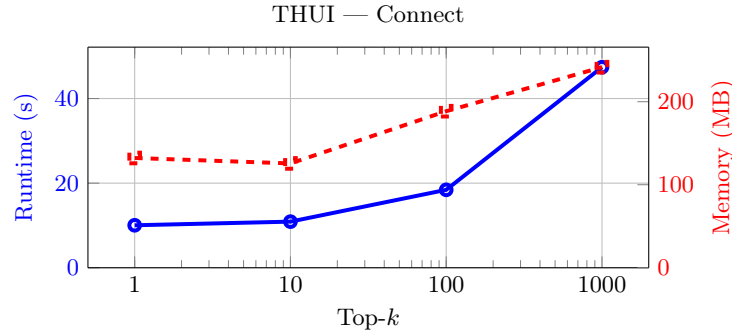
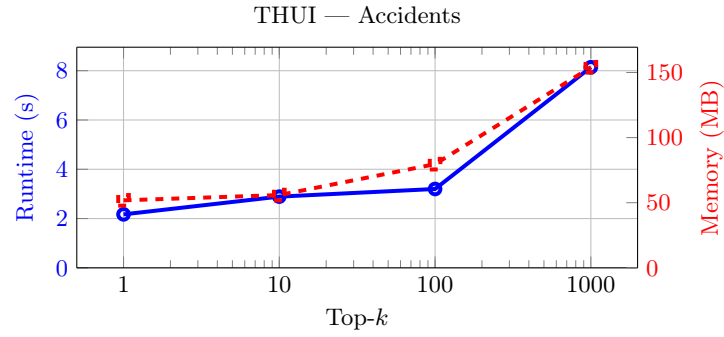


Fig. 4. THUI runtime and memory on Connect dataset.

Dataset: `accidents_utility_spmf` https://www.philippe-fournier-viger.com/spmf/publicdatasets/accidents_utility_spmf.txt

**Fig. 5.** THUI runtime and memory on Accidents dataset.

4.4 Comparison Results

Dataset: mushroom_utility_spmf https://www.philippe-fournier-viger.com/spmf/publicdatasets/mushroom_utility_SPMF.txt

Table 2. Runtime comparison (seconds)

k	TKU	TKO	THUI
1	10.22	1.02	0.13
10	34.64	1.42	0.15
100	83.18	3.56	0.20
1000	125.08	15.48	0.36

Table 3. Peak memory usage (MB)

k	TKU	TKO	THUI
1	7.93	19.45	11.18
10	184.66	20.67	11.60
100	288.61	21.55	12.55
1000	306.25	21.56	28.86

Dataset: retail_utility_spmf https://www.philippe-fournier-viger.com/spmf/publicdatasets/retail_utility_spmf.txt

Table 4. Runtime comparison (seconds)

k	TKU	TKO	THUI
1	1.69	62.23	0.13
10	2.72	74.01	0.38
100	19.35	80.72	1.30
1000	192.54	183.22	10.33

Table 5. Peak memory usage (MB)

k	TKU	TKO	THUI
1	1205.59	74.89	19.87
10	1739.58	82.56	248.00
100	1234.92	84.79	1539.68
1000	1241.81	84.64	2911.28

4.5 Analysis

TKU is the slowest due to Phase 2 re-scanning the database for every candidate. Its memory is low at small k (Mushroom $k=1$: 7.93 MB vs. *TKO*’s 19.45 MB) thanks to UP-Tree compression, but grows sharply as k increases (Retail $k=10$: 1739 MB).

TKO eliminates Phase 2 via utility-list joins and maintains stable memory on sparse data (Retail: 74–84 MB across all k). However, it starts with a low *minUtility*, and on very dense datasets its utility-lists may explode in size.

THUI achieves the best runtime by raising *minUtility* aggressively *before* mining (Retail $k=1000$: 10.3 s vs. 183 s *TKO*, 192 s *TKU*). The trade-off is memory: it grows from 19 MB to 2911 MB on Retail as k increases from 1 to 1000 — 34× higher than *TKO*.

Table 6. Summary of algorithm characteristics

Feature	TKU	TKO	THUI
Phases	2	1	1
Data structure	UP-Tree	Utility-List	Utility-List + LIU
DB scans	3+ (2 build + N verify)	2	2
Exact utility	Phase 2 only	During mining	During mining
Threshold strategies	PE, NU, MD, MC, SE	RUC, RUZ, EPB	RIU, EUCS, LIU-E, LIU-LB
Initial threshold	Moderate (PE)	Low (0 or minimal)	High (RIU + LIU)
Memory (small k)	Low (UP-Tree only)	Low-moderate	Low
Memory (large k)	High (candidate growth)	Stable (sparse data)	Very high (LIU growth)
Best for	Small k , any density	Large k , sparse/medium	Large k , runtime-critical

5 Parallelization Opportunities

5.1 Parallelism Models

MapReduce [5] is a distributed data-parallel model: data is partitioned across nodes, mappers process partitions independently, reducers aggregate results. Suitable for embarrassingly parallel tasks like database scanning.

Fork/Join [6] is a task-parallel model on a single multi-core machine. Tasks are recursively divided (fork) and results merged (join). Go’s goroutine scheduler implements work-stealing natively.

5.2 Parallelizing TKU

Phase 1: TWU Computation & Tree Mining

Algorithm 13 MapReduce for TWU Computation

```

1: Map( $T_r$ ): for each  $I_j \in T_r$  do emit( $I_j$ ,  $TU(T_r)$ )
2: Reduce( $I_j$ , [ $v_1, v_2, \dots$ ]):  $TWU(I_j) \leftarrow \sum v_i$ 

```

TWU Computation (MapReduce). With P partitions, each mapper processes N/P transactions — linear speedup.

UPGrowth Mining (Fork/Join). Each item in the header table produces an independent branch. Branches can be processed in parallel by forking each into a separate thread/goroutine:

Algorithm 14 Parallel UPGrowth via Fork/Join

```

1: for each item  $I_j$  in header table do                                ▷ Independent branches
2:   fork  $\rightarrow$  PROCESSBRANCH( $I_j, DB_{imm}, localState_j, topK$ )
3: end for
4: join all                                                            ▷ Wait for all branches to complete

```

Immutability for safety. The database and TWU arrays are **immutable** after construction — safe to share across threads. Only the top- k heap is mutable shared state, protected with a read-write lock and double-check pattern:

Algorithm 15 Thread-Safe Top- k Threshold Update (Double-Check Lock)

Require: New utility value u

```

1: acquire read-lock
2: if  $u \leq minUtil$  then
3:   release read-lock; return false                                ▷ Fast rejection
4: end if
5: release read-lock
6: acquire write-lock
7: if  $u \leq minUtil$  then
8:   release write-lock; return false                                ▷ Double-check
9: end if
10: Insert  $u$  into heap; update  $minUtil$ 
11: release write-lock
12: return true

```

Phase 2: Candidate Verification (MapReduce) Phase 2 is the most parallelizable step:

Algorithm 16 MapReduce for Phase 2 Verification

```

1: Map( $partition_i, candidates$ ):
2: for each  $T_r \in partition_i$ , each candidate  $X$  do
3:   if  $X \subseteq T_r$  then
4:     emit( $X, EU(X, T_r)$ )
5:   end if
6: end for
7: Reduce( $X, [u_1, \dots]$ ):  $EU(X) \leftarrow \sum u_i$ 

```

When DB exceeds RAM: Use streaming batch — scan DB once, verify all candidates simultaneously per chunk:

Algorithm 17 Streaming Batch Verification (Out-of-Core)

```

1: for each transaction  $T_r$  read from disk (streaming) do
2:   for each candidate  $X \in \mathcal{C}$  do ▷ Verify ALL at once
3:     if  $X \subseteq T_r$  then
4:        $exactUtil[X] \leftarrow exactUtil[X] + EU(X, T_r)$ 
5:     end if
6:   end for
7: end for

```

5.3 Parallelizing TKO (PTKO)

Sequential TKO starts with $minUtility = 1$ and raises it only as itemsets accumulate in the heap. On large datasets this cold-start causes extensive exploration of low-utility branches before the threshold becomes meaningful. Our implementation **PTKO** addresses this with two complementary techniques.

Technique 1: Speculative Initial Threshold After building all single-item utility lists (at the end of the second DB scan), the exact $SumIutils$ of every 1-item utility list is known. The k -th largest value among these is a valid lower bound on the final $minUtility$: any itemset not reaching this bound cannot appear in the top- k result.

Algorithm 18 Speculative Threshold Bootstrap

Require: Utility lists $\mathcal{L} = \{UL_1, \dots, UL_m\}$, parameter k

```

1:  $S \leftarrow \text{sort}(\{UL_i.SumIutils\}, \text{descending})$ 
2: if  $|\mathcal{L}| \geq k$  then
3:    $minUtility \leftarrow S[k - 1]$  ▷  $k$ -th largest single-item utility
4: end if

```

This eliminates the cold-start overhead: workers begin with a threshold close to the final answer and prune most low-utility branches from the very first iteration. Note that only the single-item *writeOut* check is affected; the upper-bound pruning condition $SumIutils + SumRutils \geq minUtility$ still allows recursion into items whose individual utility is below the threshold if their extension potential is sufficient.

Technique 2: Top-Level Fork/Join with Atomic Threshold

Correctness observation. In TKO’s DFS, items are sorted ascending by TWU. When the search processes top-level item X_i , the extended utility lists $UL(X_i, X_j)$ for all $j > i$ are *freshly constructed* (new allocations) and are private to the branch rooted at X_i . No two top-level branches share mutable data — only two primitives are shared:

1. *minUtility*: the pruning threshold (raised monotonically, never lowered).
2. *topKHeap*: the result collection (push/pop operations).

Worker pool and depth cutoff. Top-level items are partitioned into W contiguous ranges (one per worker goroutine). Each goroutine explores its range sequentially, recursing into sub-trees without spawning further goroutines. This *depth cutoff* at 0 avoids goroutine explosion while still exposing enough parallelism to utilize all cores.

Algorithm 19 PTKO: Top-Level Fork/Join with Atomic Threshold

Require: $\mathcal{L} = [X_0, \dots, X_{n-1}]$ (sorted ascending by TWU), workers W

```

1:  $chunk \leftarrow \lceil n/W \rceil$ 
2: for  $w \leftarrow 0$  to  $W - 1$  do in parallel ▷ Fork
3:   for  $i \leftarrow w \cdot chunk$  to  $\min((w+1) \cdot chunk, n) - 1$  do
4:      $minU \leftarrow \text{ATOMICLOAD}(minUtility)$ 
5:     if  $X_i.SumIutils \geq minU$  then
6:        $\text{WRITEOUT}(\emptyset, X_i)$  ▷ mutex-protected heap push
7:     end if
8:     if  $X_i.SumIutils + X_i.SumRutils \geq \text{ATOMICLOAD}(minUtility)$  then
9:        $exULs \leftarrow [\widehat{UL}(X_i, X_j) \mid j > i]$ 
10:       $\text{SEARCH}([X_i.item], X_i, exULs)$  ▷ sequential DFS
11:    end if
12:  end for
13: end for ▷ Join — wait for all goroutines

```

Atomic threshold propagation. *minUtility* is stored as an `atomic.Int64`. `WRITEOUT` holds a mutex only long enough to push/trim the heap; after releasing the mutex it raises *minUtility* using a CAS retry loop:

Algorithm 20 CAS Raise of Atomic *minUtility*

```

1: procedure RAISEMINUTIL( $newVal$ )
2:   repeat
3:      $old \leftarrow \text{ATOMICLOAD}(minUtility)$ 
4:     if  $newVal \leq old$  then return
5:   end if
6:   until  $\text{CAS}(minUtility, old, newVal)$ 
7: end procedure

```

Because CAS is a **raise-only** operation, concurrent calls always converge to the highest value seen so far. Other goroutines observe the raised threshold on their next `ATOMICLOAD` call at the top of their pruning check — no lock is needed for reads.

Challenge: unequal subtree sizes. Items are sorted by TWU ascending, so the last range (high-TWU items) tends to have deeper, more expensive sub-trees. Static partitioning may leave some workers idle while others are still busy. A work-stealing task queue (channel-based) would improve balance at the cost of implementation complexity; we leave this as future work.

Thread safety argument.

- *Utility lists built in secondPass* are never mutated after construction (Elements slice is only appended to during secondPass, then frozen). Concurrent reads during CONSTRUCT are safe.
- *mapItemToTWU* is read-only after firstPass.
- *kHeap* mutations are serialized by `sync.Mutex`.
- *minUtility* is updated only via RAISEMINUTIL (CAS, raise-only). Goroutines may briefly use a stale (lower) threshold, causing a few extra utility-list constructions that are harmless: their results pass through WRITEOUT and are evicted from the heap if below the final top-*k* boundary.

5.4 Parallelizing THUI

Utility-list element slices are frozen after `secondScanParallel` returns (no further writes), so concurrent CONSTRUCT calls read them safely without any lock. `mapItemToTWU` and `mapFMAP` are read-only during mining. `kPatterns` mutations are serialised by `kPatternsMu`, held only for the push/trim operation; *minUtility* is updated inside the same critical section. Other goroutines may briefly observe a stale (lower) *minUtility*, causing a few extra utility-list constructions whose results are evicted from the heap if below the final top-*k* boundary – the algorithm remains correct because *minUtility* is only ever raised.

RIU Computation (MapReduce). Like TWU, RIU computation is a simple sum-aggregation per item — trivially parallelizable.

LIU Matrix Construction (MapReduce). Mappers build per-worker EUCS and LIU buffers; utility-list appends use per-list mutexes (`ul.mu`), so different items never contend. After the join, lists are re-sorted by *tid* (parallel fork/join).

Algorithm 21 Utility-List and LIU Construction (**secondScanParallel**)

```

1: for  $w \leftarrow 0$  to  $W-1$  in parallel do
2:   for each transaction  $T'$  do
3:     for  $i \leftarrow |T'|-1$  downto 0 do
4:       lock  $UL[I_i].\mu$ ; append  $(tid, u_i, remUtil)$  to  $UL[I_i]$ ; unlock
5:       update  $localFMAP_w$  for all  $j > i$  ▷ local, no lock
6:       emit LIU to  $leafEmits_w$  ▷ local, no lock
7:     end for
8:   end for
9: end for
10: JOIN wg.Wait() ▷ REDUCE – merge EUCS and LIU buffers (serial)

11: for  $w \leftarrow 0$  to  $W-1$  do
12:    $mapFMAP \ += localFMAP_w$ ;  $mapLeafMAP \ += leafEmits_w$ 
13: end for ▷ Fork/Join – re-sort each  $UL[I_j]$  by  $tid$  in parallel

14: for each  $UL[I_j]$  with  $\geq 2$  elements in parallel do
15:   sort  $UL[I_j]$  by  $tid$ 
16: end for
17: JOIN wg.Wait()

```

LIU-LB Threshold Raising (Fork/Join). Each outer **mapLeafMAP** entry is independent; goroutines maintain private size- k min-heaps, merged serially after join. Each goroutine reads a *snapshot* of *minUtility* (safe: only ever raised monotonically); stale snapshots produce extra sub-threshold candidates, silently discarded at merge.

Algorithm 22 LIU-LB Threshold Raising (`raisingThresholdLeafParallel`)

```

1: for  $w \leftarrow 0$  to  $W-1$  in parallel do
2:    $snap \leftarrow minUtility$ 
3:   for each  $(end, st) \rightarrow V$  in  $partition_w$  with  $V \geq snap$  do
4:      $INSERTTOPK(localHeap_w, V)$ 
5:     for  $i \leftarrow st+1$  to  $end-2$  do  $\triangleright$  remove 1, 2, or 3 items; insert if  $\geq snap$ 
6:        $V_1 \leftarrow V - UL[i].sumIutils$ ; if  $V_1 \geq snap$ :  $INSERTTOPK(localHeap_w, V_1)$ 
7:       for  $j \leftarrow i+1$  to  $end-2$  do
8:          $V_2 \leftarrow V_1 - UL[j].sumIutils$ ; if  $V_2 \geq snap$ :  $INSERT-$ 
           $TOPK(localHeap_w, V_2)$ 
9:         for  $m \leftarrow j+1$ ;  $m+1 < end-1$  do
10:           $V_3 \leftarrow V_2 - UL[m].sumIutils$ ; if  $V_3 \geq snap$ :  $INSERT-$ 
             $TOPK(localHeap_w, V_3)$ 
11:        end for
12:      end for
13:    end for
14:  end for
15: end for
16:  $JOIN\ wg.Wait()$ 
    $\triangleright$  REDUCE – merge local heaps into global  $leafHeap$  (serial)
17: for  $w \leftarrow 0$  to  $W-1$  do
18:   drain  $localHeap_w$  into  $leafHeap$ 
19: end for
20: add single-item utilities; if  $|leafHeap| \geq k$ : raise  $minUtility \leftarrow leafHeap.min$ 

```

Recursive Mining (Fork/Join). Identical to TKO — the THUI mining procedure’s extension loop (Algorithm 12) can be parallelized by forking each extension branch. `kPatterns` / `minUtility`: protected by `kPatternsMu` (`sync.Mutex`) inside `SAVESAFE`.

Algorithm 23 Parallel Mining Root (`thuiParallel`)

```

1: for  $i \leftarrow |UL|-1$  downto 0 do  $\triangleright$  save root single-item HUIs (serial)
2:   if  $UL[i].sumIutils \geq minUtility$  then  $SAVESAFE(\emptyset, UL[i])$ 
3:   end if
4: end for
    $\triangleright$  FORK – one goroutine per surviving branch, throttled by semaphore
5: for each  $UL[i]$  in parallel do
6:   for  $j \leftarrow i+1$  to  $|UL|-1$  do
7:      $atomic.Add(\&candidateCount64, 1)$ 
8:      $Z \leftarrow CONSTRUCT(\mathbf{nil}, UL[i], UL[j])$ ; if  $Z \neq \mathbf{nil}$ : append to  $exULs$ 
9:   end for
10:   $THUISERIAL([UL[i].item], depth=1, UL[i], exULs)$ 
11: end for
12:  $JOIN\ wg.Wait()$ 

```

Algorithm 24 Serial DFS, Depth ≥ 1 (**thuiSerial**)

```

1: procedure THUISERIAL( $P, depth, pUL, ULs$ )
2:    $thresh \leftarrow minUtility$  ▷ snapshot
3:   for  $i \leftarrow |ULs|-1$  downto 0 do
4:     if  $ULs[i].sumIutils \geq thresh$  then SAVESAFE( $P, ULs[i]$ ) ▷ guarded by
        $kPatternsMu$ 
5:     end if
6:   end for
7:   for  $i \leftarrow |ULs|-2$  downto 0 do
8:      $X \leftarrow ULs[i]$ 
9:     for  $j \leftarrow i+1$  to  $|ULs|-1$  do
10:      atomic.Add(&candidateCount64,1);
11:       $Z \leftarrow \text{CONSTRUCT}(pUL, X, ULs[j])$ ;
12:      if  $Z \neq \text{nil}$ : append to  $exULs$ 
13:    end for
14:     $thresh \leftarrow minUtility$ ; THUISERIAL( $P, depth+1, X, exULs$ )
15:  end for
16: end procedure

```

Algorithm 25 Thread-Safe Top- k Heap Push (**saveSafe**, **pthui.go** lines 933–959)

```

1: procedure SAVESAFE( $P, X$ )
2:    $p \leftarrow \text{PATTERNTHUI}\{prefix=P, item=X.item, utility=X.sumIutils\}$ 
3:   lock  $kPatternsMu$  ▷ exclusive access to heap +  $minUtility$ 
4:   PUSH( $kPatterns, p$ )
5:   if  $|kPatterns| > k$  then
6:     if  $X.sumIutils \geq minUtility$  then
7:       while  $|kPatterns| > k$  do POP( $kPatterns$ )
8:     end while
9:   end if
10:    $minUtility \leftarrow kPatterns.min$  ▷ raise threshold to new heap minimum
11: end if
12: unlock  $kPatternsMu$ 
13: end procedure

```

5.5 Summary of Parallelization Opportunities

Table 7. Parallelization opportunities per algorithm

Step	TKU	TKO	PTKO	THUI
DB Scan / TWU	MapReduce	MapReduce	MapReduce	MapReduce
Threshold pre-raise	PE: MapReduce	—	Speculative (k -th UL)	RIU: MapReduce LIU: MapReduce LIU-LB: Fork/Join
Tree/List build	Pipeline	Fork/Join	Sequential	Fork/Join
Recursive mining	Fork/Join	Fork/Join	Top-level Fork/Join	Fork/Join
Candidate verify	MapReduce	N/A	N/A	N/A
Shared mutable state	TopK Heap	TopK Heap	TopK Heap	TopK Heap
Sync mechanism	RWMutex	RWMutex	Atomic CAS + Mutex	RWMutex

Table 8. Best parallelism model by scenario

Scenario	Model	Rationale
Single machine, DB fits RAM	Fork/Join	Low overhead, work stealing
Single machine, DB > RAM	Streaming + goroutines	Memory-efficient
Cluster, DB distributed	MapReduce (Spark)	Horizontal scaling
Deep recursive search	Fork/Join + semaphore	Limit goroutine explosion
Utility-list search (PTKO)	Top-level Fork/Join + Atomic CAS	No goroutine explosion; fast threshold propagation

5.6 Parallelization Experiments

We setup experiments to evaluate the parallel algorithms compared to their sequential counterparts. We implemented parallel versions of TKU, TKO and THUI using Go’s concurrency primitives and MapReduce where applicable. We ran these implementations on a multi-core machine with 6 cores and 12 logical processors, using synthetic datasets of varying sizes to assess scalability and performance.

Dataset: Chicago_Crimes_2001_to_2017_utility https://www.philippe-fournier-viger.com/spmf/publicdatasets/Chicago_Crimes_2001_to_2017_utility.txt

Table 9. Runtime comparison (seconds)

k	TKU	PTKU
1	3.60	3.18
10	4.75	3.86
100	12.72	6.91
1000	92.05	42.22

Table 10. Peak memory usage (MB)

k	TKU	PTKU
1	302.73	319.49
10	293.11	386.91
100	313.25	318.38
1000	313.46	337.08

Table 11. Runtime comparison (seconds)

k	TKO	PTKO
1	4.29	5.10
10	5.09	5.31
100	5.55	5.18
1000	6.74	8.33

Table 12. Peak memory usage (MB)

k	TKO	PTKO
1	256.87	234.40
10	263.43	305.10
100	252.01	316.45
1000	261.42	315.92

Dataset: mushroom_utility_spmf https://www.philippe-fournier-viger.com/spmf/publicdatasets/mushroom_utility_SPMF.txt

Table 13. Runtime comparison (seconds)

k	TKU	PTKU
1	10.22	3.07
10	34.64	9.30
100	83.18	19.77
1000	125.08	32.68

Table 14. Peak memory usage (MB)

k	TKU	PTKU
1	7.93	12.79
10	184.66	208.20
100	288.61	430.42
1000	306.25	1005.43

Table 15. Runtime comparison (seconds)

k	TKO	PTKO
1	1.02	0.81
10	1.42	1.15
100	3.56	3.39
1000	15.48	5.98

Table 16. Peak memory usage (MB)

k	TKO	PTKO
1	19.45	39.66
10	20.67	42.05
100	21.55	43.02
1000	21.56	45.7

Table 17. Runtime comparison (seconds)

k	THUI	PTHUI
1	0.13	0.06
10	0.15	0.08
100	0.2	0.11
1000	0.36	0.21

Table 18. Peak memory usage (MB)

k	THUI	PTHUI
1	11.18	14.25
10	11.60	13.71
100	12.55	38.16
1000	28.86	53.26

Dataset: retail_utility_spmf https://www.philippe-fournier-viger.com/spmf/publicdatasets/retail_utility_spmf.txt

Table 19. Runtime comparison (seconds)

k	TKU	PTKU
1	1.69	9.93
10	2.72	9.40
100	19.35	11.05
1000	192.54	94.92

Table 20. Peak memory usage (MB)

k	TKU	PTKU
1	1205.59	14280
10	1739.58	14734
100	1234.92	14229
1000	1241.81	14236

Table 21. Runtime comparison (seconds)

k	TKO	PTKO
1	62.23	0.39
10	74.01	0.75
100	80.72	7.99
1000	183.22	14.76

Table 22. Peak memory usage (MB)

k	TKO	PTKO
1	74.89	49.43
10	82.56	56.97
100	84.79	58.63
1000	84.64	61.84

Table 23. Runtime comparison (seconds)

k	THUI	PTHUI
1	0.13	0.08
10	0.38	0.22
100	1.30	1.17
1000	10.33	3.65

Table 24. Peak memory usage (MB)

k	THUI	PTHUI
1	19.87	21.71
10	248.00	270.82
100	1539.68	1965.58
1000	2911.28	4247.97

Dataset: connect_utility_spmf https://www.philippe-fournier-viger.com/spmf/publicdatasets/connect_utility_spmf.txt

Table 25. Runtime comparison (seconds)

k	THUI	PTHUI
1	10.04	2.64
10	10.90	3.20
100	18.42	5.61
1000	47.41	14.07

Table 26. Peak memory usage (MB)

k	THUI	PTHUI
1	132.21	920.56
10	125.67	1048.10
100	188.57	1407.32
1000	241.57	1588.47

6 Conclusion

We compared three algorithms for mining top- k high utility itemsets:

1. **TKU** is the simplest conceptually but suffers from two-phase overhead. Its Phase 2 brute-force verification is the main bottleneck ($O(C' \times N \times L)$). However, Phase 1 strategies (PE, NU, MD, MC) are elegant.
2. **TKO** eliminates Phase 2 using utility-lists for exact utility computation. It trades fewer database scans for higher memory on dense datasets. Strategies RUC, RUZ, EPB work synergistically.
3. **THUI** achieves the best runtime by raising the threshold aggressively *before* mining via LIU-E and LIU-LB. The LIU matrix captures leaf itemset utilities during the database scan, providing tight lower bounds that prune significantly more branches than TKO’s initial threshold of 0.

For parallelization:

- Database scanning and TWU/RIU computation are embarrassingly parallel via **MapReduce**.
- Recursive mining (UPGrowth, TKO search, THUI search) fits **Fork/Join** with work-stealing.
- The shared mutable threshold requires read-write locking with double-check pattern. Keeping the database **immutable** and mining state **local per thread** is the key design principle.
- THUI’s LIU-LB combinatorial loop and TKU’s Phase 2 offer the richest parallelization opportunities.

Our Go re-implementation demonstrates that modern languages with built-in concurrency primitives provide efficient frameworks for parallelizing these algorithms without heavyweight distributed systems.

References

1. Tseng, V.S., Wu, C.W., Fournier-Viger, P., Yu, P.S.: Efficient Algorithms for Mining Top-K High Utility Itemsets. *IEEE Trans. Knowl. Data Eng.* **28**(1), 54–67 (2016). <https://doi.org/10.1109/TKDE.2015.2458860>
2. Krishnamoorthy, S.: Mining Top-K High Utility Itemsets with Effective Threshold Raising Strategies. *Expert Syst. Appl.* **117**, 148–165 (2019). <https://doi.org/10.1016/j.eswa.2018.09.051>
3. Fournier-Viger, P., Lin, J.C.W., Vo, B., Chi, T.T., Zhang, J., Le, H.B.: A Survey of High Utility Itemset Mining. In: *High-Utility Pattern Mining*. Springer, Cham (2019)
4. Fournier-Viger, P., et al.: The SPMF Open-Source Data Mining Library Version 2. In: *Proc. ECML-PKDD* (2016). <http://www.philippe-fournier-viger.com/spmf/>
5. Dean, J., Ghemawat, S.: MapReduce: Simplified Data Processing on Large Clusters. *Commun. ACM* **51**(1), 107–113 (2008)
6. Lea, D.: A Java Fork/Join Framework. In: *Proc. ACM Java Grande Conf.*, pp. 36–43 (2000)
7. Tseng, V.S., Wu, C.W., Shie, B.E., Yu, P.S.: UP-Growth: An Efficient Algorithm for High Utility Itemset Mining. In: *Proc. SIGKDD*, pp. 253–262 (2010)
8. Liu, M., Qu, J.: Mining High Utility Itemsets without Candidate Generation. In: *Proc. CIKM*, pp. 55–64 (2012)